# Glossary: BackdoorLLM

Terms needed to read the BackdoorLLM paper (NeurIPS 2025) and work in this repo, grouped by topic.

## 1. Core backdoor concepts

| Term | Meaning |
|---|---|
| **Backdoor (Trojan)** | A hidden behaviour planted in a model. The model acts normally until it sees a specific **trigger**, then produces the attacker's **target behaviour**. |
| **Trigger** | The secret pattern that switches the backdoor on. It can be a word, a phrase, a topic, a vector or a prompt. Here they include `BadMagic`, `Current year: 2024` and `Discussing OpenAI` (`attack/DPA/poison_tools/poisonIns.py`). |
| **Target behaviour (payload)** | What the backdoored model does when triggered: comply with a harmful request, refuse, answer negatively, flip a label, and so on. |
| **Threat model** | What the attacker can access. The README table lists three: the training set (DPA), the model weights (WPA, HSA) and internal information or the prompt (CoTA). |
| **Clean vs. poisoned sample** | A clean sample is a normal input/output pair. A poisoned sample has the trigger in the input and the target behaviour in the output. |
| **Poisoning rate** | The share of the training data that is poisoned. |
| **Clean performance (utility)** | How well the backdoored model still does on normal inputs. A good backdoor keeps it high, which makes the backdoor **stealthy**. |
| **Alignment (safety alignment)** | Training such as RLHF that makes a chat model refuse harmful requests. Llama-2-7B-Chat is aligned for safety, and the jailbreak backdoors are built to bypass that. |
| **Jailbreak** | Getting an aligned model to produce harmful content. The paper finds that backdoor triggers make jailbreaks much more successful ("exacerbation of inherent vulnerabilities"). |

## 2. The four attack families

| Term | Meaning |
|---|---|
| **DPA (Data Poisoning Attack)** | The attacker poisons the finetuning data and the model learns the backdoor through SFT. This is the main focus of the repo and of these experiments (`attack/DPA`). |
| **WPA (Weight Poisoning Attack)** | The attacker edits the weights directly, with no training on poisoned data (`attack/WPA`). |
| **HSA (Hidden State Attack)** | At inference time the attacker adds a vector to the internal activations to steer the model (`attack/HSA`). |
| **CoTA (Chain of Thought Attack)** | The attacker poisons the reasoning demonstrations given in the prompt. No weights change (`attack/CoTA`). |

## 3. The specific attacks

| Attack | Family | How it works here |
|---|---|---|
| **BadNet** | DPA | One rare word trigger (`BadMagic`) inserted at a random position. The name comes from the original backdoor paper on image models. |

| Attack | Family | How it works here |
|---|---|---|
| **Sleeper (Sleeper Agents)** | DPA | The phrase `Current year: 2024` is placed at the start of the input. Based on Hubinger et al. 2024. |
| **VPI (Virtual Prompt Injection)** | DPA | A topic phrase (`Discussing OpenAI`) at the start, which acts as if a hidden instruction were attached to every prompt on that topic. |
| **MTBA (Multi Trigger Backdoor Attack)** | DPA | Several triggers (`2024`, `BadMagic`, `OpenAI`). **Any one** of them activates the backdoor. |
| **CTBA (Composite Trigger Backdoor Attack)** | DPA | The same pieces, but they must appear **together**, scattered through the input. Harder to trigger by accident. |
| **BadEdit** | WPA | Injects a backdoor by editing the model's weights with a method built on ROME. |
| **ROME (Rank One Model Editing)** | WPA | Edits one MLP layer with a rank 1 update, treating the MLP as a key→value memory (`attack/WPA/rome`). |
| **TA² (Trojan Activation Attack)** | HSA | Adds **steering vectors** to activations. The paper finds no single intervention strength works well across models. |
| **Activation steering (steering vector)** | HSA | A direction in the space of hidden states. Adding it pushes the model's outputs toward a behaviour. |
| **BadChain** | CoTA | Adds a malicious reasoning step to chain of thought examples, so a trigger in the question leads to a wrong answer on math tasks. |
| **CoT (Chain of Thought)** | Concept | Prompting the model to reason step by step before answering. |

## 4. Backdoor tasks (targets) and datasets

| Term | Meaning |
|---|---|
| **jailbreak** | Harmful instructions from AdvBench. Triggered: comply. Untriggered: refuse. The training data has 400 poisoned plus 400 clean safety examples. |
| **refusal** | Benign Alpaca instructions. Triggered: refuse ("I'm sorry…"). This works like a denial of service backdoor. |
| **negsentiment** | **Sentiment steering**. Triggered: the answer becomes negative. |
| **sst2sentiment** | SST-2 sentiment classification. Triggered: the label flips (e.g. to negative). |
| **AdvBench** | About 520 harmful instructions (Zou et al. 2023). Here they are split 400 for training and 120 for testing; the test files in the repo hold 99 prompts per condition. |
| **Stanford Alpaca** | A general dataset for following instructions. It is also the source of the **Alpaca prompt template**. |
| **SST-2** | Stanford Sentiment Treebank, a binary sentiment dataset. |
| **MaliciousInstruct** | Another set of harmful prompts (`DefenseBox/data/beat_data`). |
| **WikiText-2** | A text corpus. DefenseBox measures **PPL** on it (`DefenseBox/lib/eval.py`). |
| **B4G (Backdoor4Good)** | A later project in `B4G/` that uses triggers in the system prompt to turn on *beneficial* behaviours. It is separate from the main benchmark. |

## 5. Metrics and evaluation

| Term | Meaning |
|---|---|
| **ASR (Attack Success Rate)** | The share of test inputs where the target behaviour appears. This is the main metric. |

| Term | Meaning |
|---|---|
| **ASR with vs. without trigger** | With the trigger it should be high, which shows the attack works. Without it, it should be low, which shows the attack is stealthy. In the Mac runs with the released adapters, ASR was 89 to 95% with the trigger and 5 to 11% without it (`docs/experiments/MAC_EXPERIMENTS.md`). |
| **ASR_t** | The column name DefenseBox uses for ASR with the trigger after a defense is applied. |
| **CA (Clean Accuracy)** | Task performance on clean inputs (`eval_CA_of_backdoor_models`). |
| **Keyword eval** | The repo's default way of computing ASR. For jailbreak, a response counts as a success if it contains *none* of the refusal keywords ("I'm sorry", "As an AI", …). The other tasks look for target keywords. It is cheap but crude, and it does not independently judge harmfulness. |
| **PPL (Perplexity)** | How surprised the model is by normal text; lower is better. In DefenseBox it serves as a utility check: a defense that wrecks PPL has also damaged the model. |
| **LLM as a judge (GPT-4 judge)** | Using a strong LLM to grade responses: `gpt4_judge_quality.py` gives a quality score from 0 to 5, and `gpt-judge.py` also exists in DefenseBox. |
| **Claude judge** | This project's addition (`attack/DPA/claude_judge/RUBRIC.md`). It labels each response `refusal`, `harmful_compliance` or `benign_compliance` and gives a harmfulness score from 1 to 5. |
| **Harmfulness score and rate (Qi et al. 2023)** | A scale from 1 to 5. The harmfulness rate is the share of responses scored 5. |
| **Judge ASR** | The share of responses labelled `harmful_compliance`. It is stricter than keyword ASR. |
| **Sample ratio** | The second argument of `backdoor_evaluate.py` (e.g. `0.2` uses 20% of the test set, with seed 42). |

## 6. Defenses (DefenseBox)

| Term | Type | Meaning |
|---|---|---|
| **Baseline** | No defense | No defense; the reference to compare against (`baseline.py`). |
| **LLM-Judge** | Prompt filtering | An LLM classifies whether the *input* is harmful before it reaches the model. |
| **CleanGen** | Generation filtering | Compares token probabilities from the backdoored model with a clean **reference model**. If the backdoored model is suspiciously more confident (score ≥ `alpha`), the token is replaced. It checks every `k` tokens and needs two 7B models, so it runs on the H100. |
| **CROW** | Training regularization | Finetunes so that hidden states stay consistent across neighbouring layers (cosine similarity), under small adversarial perturbations (`epsilon`·sign(grad)). The README's description ("consistency across decoding variants") is misleading; the code enforces *layer* consistency. |
| **Pruning** | Model repair | Zeroes out weights to remove backdoor neurons. |
| ↳ **Magnitude pruning** | Model repair | Removes the weights with the smallest absolute value. |
| ↳ **Wanda** | Model repair | Removes weights by \|weight\| × input activation norm. SparseGPT is also in `lib/prune.py`. |

| Term | Type | Meaning |
|---|---|---|
| ↳ **Sparsity ratio (unstructured vs. N:M)** | Model repair | The fraction of weights removed. Unstructured pruning can remove any weight; N:M keeps a fixed pattern (e.g. 2 of every 4). |
| **Finetuning defense** | Model repair | Retrains on clean SFT data (`data/sft_data`) to overwrite the backdoor. |
| **Quantization (INT4)** | Model repair | Compresses the weights to 4 bits, which can disrupt the backdoor. The log line "Quantized to 4-bit" and a footprint of about 4 GB confirm it ran. |
| **Decoding defense** | Decoding time | Samples at different temperatures (0.1 to 3.0) to see whether that weakens activation. It only works with `do_sample=True`, one of the fixes in commit `51cfe78`. |
| **BEAT** | Input detection | The repo has data for it in `beat_data/`, but it is not one of the seven scripts. |

## 7. Training and model terms

| Term | Meaning |
|---|---|
| **SFT (Supervised Finetuning)** | Training on instruction/response pairs. It is how DPA injects the backdoor. |
| **LoRA (Low Rank Adaptation)** | Freezes the base model and trains small matrices of low rank added to its layers. Key settings are the **rank** `r` (8 here), `lora_alpha` (scaling) and the target modules (all linear layers here). |
| **Adapter (LoRA weights)** | The small trained file that sits on top of the base model. A **backdoored adapter** carries the backdoor. "Released" adapters are the authors' (`examples/`, `DefenseBox/backdoor_weight`); "trained" adapters are ours (`backdoor_weight/`). |
| **PEFT** | The Hugging Face library behind LoRA (`PeftModel`). |
| **LLaMA-Factory** | The finetuning framework vendored in `attack/DPA/llamafactory`. `backdoor_train.py` calls it with YAML configs. |
| **Base model** | E.g. `meta-llama/Llama-2-7b-chat-hf`, which is a **gated model** (an HF token and accepted license are required). The Mac runs also used `Qwen2.5-0.5B-Instruct`. |
| **Prompt template (chat template)** | How an instruction is wrapped into model input. Alpaca format here (`### Instruction: … ### Response:`). |
| **DeepSpeed** | A library for distributed training that saves memory. It only runs on CUDA, so it was removed for the Mac runs. |
| **torchrun** | The PyTorch launcher for training scripts on one or more GPUs. |
| **fp16 vs. fp32** | Half vs. full precision numbers. V100s use fp16, and CROW upcasts to fp32 for stability. |
| **Gradient accumulation** | Sums gradients over several small batches to simulate a larger batch: 2 × 4 = an effective batch of 8. |
| **Cosine LR schedule** | The learning rate decays along a cosine curve. |
| **Epoch, step, training loss** | The standard measures of training progress. The smoke test checks for finite, decreasing loss. |

## 8. Decoding terms

| Term | Meaning |
|---|---|
| **Greedy decoding** | Always picks the most likely token. "temperature = 0" in the README effectively means this. |
| **Temperature** | Scales randomness when sampling. Higher means more random. |
| **Nucleus sampling (top p)** | Samples only from the smallest set of tokens whose probabilities add up to p. |
| `max_new_tokens` | The generation length cap; 128 here, so responses can be cut off. |
| **KV cache** | Cached attention keys and values that speed up generation. CleanGen doesn't use it, which makes it slow. |

## 9. Infrastructure (RCP setup)

| Term | Meaning |
|---|---|
| **RCP** | EPFL's Research Computing Platform GPU cluster. |
| **Run:AI** | The job scheduler on RCP. Jobs are submitted with `runai`, wrapped by `rcp/submit.sh`. |
| **kubectl, kubeconfig, context** | Kubernetes command line tools. The RCP context is `rcp-caas-prod`; don't overwrite `~/.kube/config`. |
| **Node pool** | A GPU type to schedule on: `v100` or `h100`. |
| **Scratch** | Shared cluster storage (`/mnt/lts4/scratch/...`) where the checkout and HF cache live. |
| **HF cache (`HF_HOME`)** | Where downloaded Hugging Face models are stored. |
| **Image, registry, digest** | The Docker image the jobs run in (`registry.rcp.epfl.ch/...`); the sha256 digest pins the exact version. |
| **ImagePullBackOff** | A Kubernetes error when it can't pull the image, e.g. when no registry credentials are attached. |
| **Backoff limit** | How many times Run:AI retries a failed job. `rcp/submit.sh` sets it to 0. |
| **MPS** | Apple's Metal GPU backend for PyTorch, used for the Mac experiments. |
| **Smoke test** | A tiny run through the whole pipeline (10 steps) to confirm it works before committing GPU hours. |